



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Organização e Arquitetura de Computadores

Prof. Rodrigo Pereira, Dr.

Última alteração: 20/08/2026

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

PROGRAMAÇÃO EM LINGUAGEM DE MONTAGEM DO MIPS (MIPS-S)

INTRODUÇÃO

- Conjunto de instruções para estudo
- MIPS-S (Simplificado) usa um subconjunto de instruções da arquitetura MIPS-2000
- Arquitetura baseada no paradigma RISC
- Ambiente de Simulação
 - MIPS Assembler and Runtime Simulator, site
 - <http://courses.missouristate.edu/KenVollmar/MARS>
 - Comando para executar o ambiente
 - `java -jar Mars4_5.jar`
- Objetivo deste material
 - Compreender conceitos básicos de programação no processador MIPS (MIPS-S)

ÍNDICE

- **Instruções Aritméticas**
 - **add/sub**
 - **Load/Store Word**
- **Instruções Lógicas**
- **Instruções de Desvio**
- **Procedimentos + Pilha**

INSTRUÇÕES BÁSICAS - A LINGUAGEM DE MÁQUINA

REVISÃO

- Para comandar um hardware de um computador é necessário **falar a linguagem dele**
- As palavras da linguagem de máquina são chamadas de **INSTRUÇÕES**
- O vocabulário é chamado de **CONJUNTO DE INSTRUÇÕES**
- **Linguagem de Montagem (Assembly)** - representação simbólica da linguagem de máquina, usando mnemônicos
- **Linguagem de Máquina** - linguagem que o processador executa diretamente, formada exclusivamente por sequências de bits (0s e 1s)

INSTRUÇÕES ARITMÉTICAS ADD/SUB

- Todo computador precisa realizar operações aritméticas - observe a notação abaixo:

add a, b, c

- Instrui o computador a: *somar as duas variáveis **c** e **b**, colocando o resultado em **a***
- Temos uma notação rígida e limitada

Categoria	Instrução	Exemplo	Significado	Comentário
Aritmética	add	add a, b, c	$a = b + c;$	sempre três operandos
	sub	sub a, b, c	$a = b - c;$	sempre três operandos

INSTRUÇÕES ARITMÉTICAS ADD/SUB (cont.)

- Com base nas instruções apresentadas observe a seguinte operação:

$$a = b + c + d + e;$$

- A sequência de instruções a seguir adiciona as variáveis **c**, **d** e **e**

```
add a, b, c    # soma de b e c, armazena em a
add a, a, d    # soma de b e c e d, armazena em a
add a, a, e    # soma de b e c e d e e, armazena em a
```

- **PRINCÍPIO DE PROJETO 1 - SIMPLICIDADE É FAVORECIDA PELA REGULARIDADE**

INSTRUÇÕES ARITMÉTICAS ADD/SUB (cont.)



Considere:

- $f = \$t0; g = \$t1; h = \$t2; i = \$t1; j = \$t3$
- Quebre a operação em C, a seguir, em instruções da linguagem de montagem:

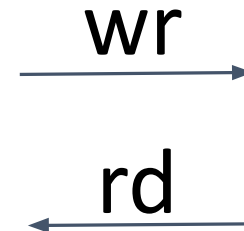
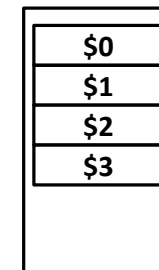
$f = (g + h) - (i + j);$

TRANSFERÊNCIA DE DADOS

LOAD WORD (LW)/STORE WORD (SW)

- Nos exemplos anteriores os valores eram únicos e simples (DADOS ESCALARES)
- Na programação real é necessário que a linguagem suporte estruturas de dados mais complexas, como *arrays*, escrita e leitura em memória
- Considerando que um processador precisa:
 - **ESCREVER DADOS** (*wr* - write data)
 - **LER DADOS** (*rd* - read data)
- A instrução para **LER** ou carregar um dado da memória para um registrador, é o **LOAD WORD** ou **LW**
- A instrução para **ESCREVER** ou salvar um dado do registrador para memória, é o **STORE WORD** ou **SW**

processador



dados	
.	.
.	.
.	.
3	100
2	10
1	101
0	1

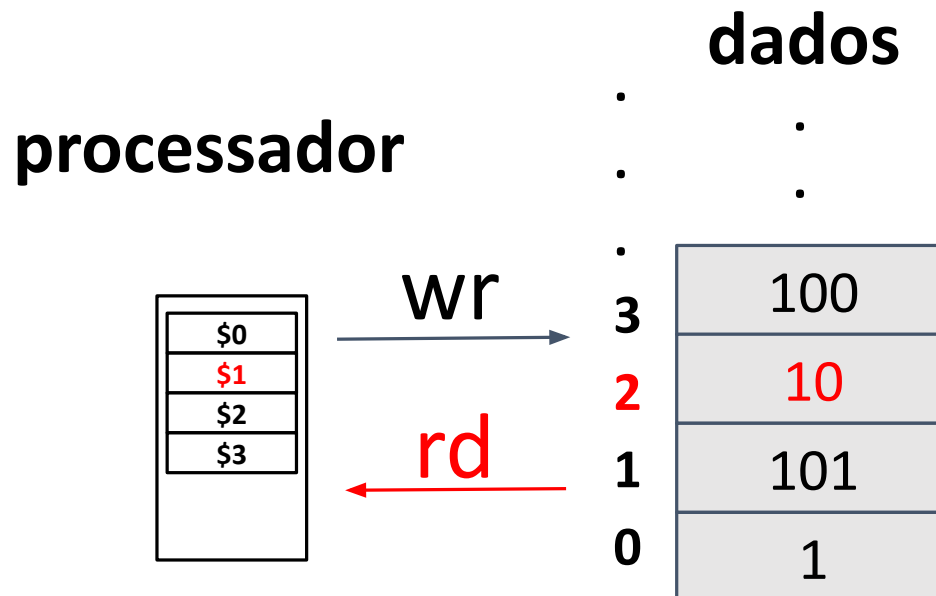
Instrução LOAD WORD

- Load word (**LW**) move/transfere uma word de dado da memória para um registrador (**\$x**) do processador

exemplo:

lw \$1, deslocamento (endereço-base) # registrador \$1 recebe o valor 10.

Equivalente a $\$s1 = \text{memória} [\text{endereço-base} + \text{deslocamento}]$



COMPILAÇÃO DE UMA ATRIBUIÇÃO COM UM OPERANDO DE MEMÓRIA

- Vamos supor que **A** seja um array de 100 palavras (cada palavra com 32 bits), e que o compilador tenha associado às variáveis **g** e **h** aos registradores **\$s1** e **\$s2**. Suponhamos também que o endereço inicial do array, ou ***endereço base***, está armazenado em **\$s3**. Traduza o seguinte comando de atribuição em C, para a linguagem de montagem do MIPS:

```
g = h + A[8] ;
```

COMPILAÇÃO DE UMA ATRIBUIÇÃO COM UM OPERANDO DE MEMÓRIA - Solução

- Primeiro é necessário transferir o que está em **A[8]** para um registrador
 - O valor buscado está no endereço **\$s3** (nosso endereço-base) somado ao deslocamento:

deslocamento:

`lw $t0, 8($s3)`

vamos assumir inicialmente esse valor

- Agora **\$t0** possui o valor que estava armazenado em **A[8]**
- A instrução precisa adicionar **h** (\$s2) com **A[8]** (\$t0) e colocar o resultado da soma em **g** (\$s1):

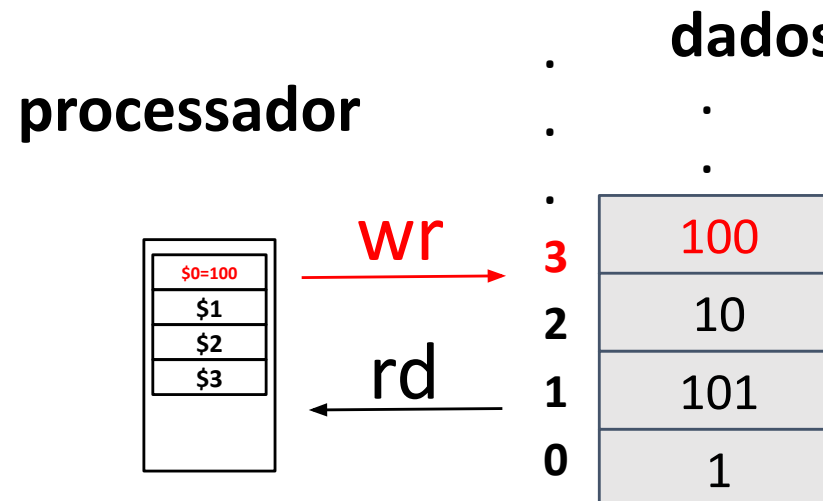
`add $s1, $s2, $t0`

Instrução STORE WORD

- Instrução complementar ao LOAD é chamada STORE
- Transfere uma cópia dos dados de um registrador para a memória

exemplo:

sw \$0, deslocamento (endereço-base) # registrador \$0 transfere o valor 100 para *memória [3]*. Equivalente a *memória[endereço-base + deslocamento] = \$0*



Compilação Usando Instruções Load/Store

- Suponha que a variável **h** esteja associada ao registrador **\$s2** e que o **endereço-base** do array **A** esteja armazenado em **\$s3**. Qual o código de montagem do MIPS para o comando de atribuição seguinte escrito em C?

A[12] = h + A[8];



Acesso a memória

COMPILAÇÃO LOAD E STORE

- Agora existem dois operando de acesso a memória
- A primeira instrução que deve ser tratada é:
 - **A[8] -> buscar na memória o conteúdo na posição $4 \times 8 = 32$**
 - **h + A[8] -> somar o conteúdo de A[8] com h**

```
lw $t0, 32($s3)
add $t0, $s2, $t0
```

 será explicado a seguir

- Por fim, a última instrução armazena no endereço A[12], utilizando o valor 48 como deslocamento em relação ao ***endereço-base***

```
sw $t0, 48($s3)
```

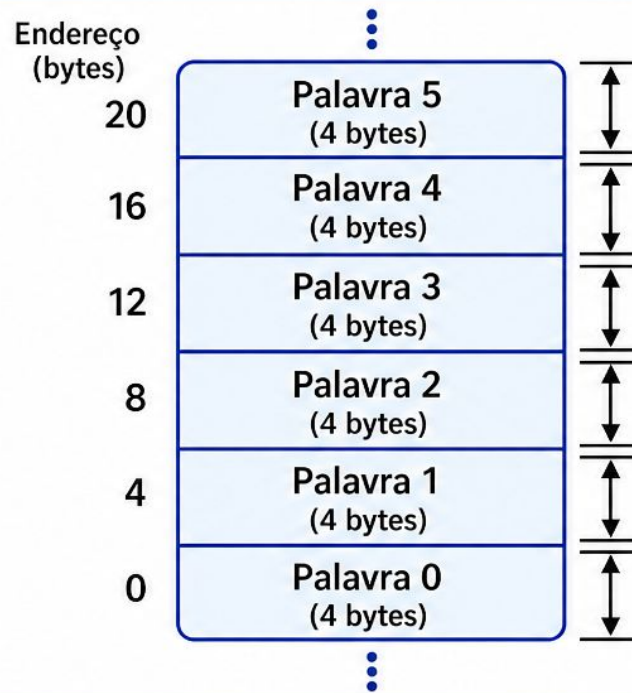
Endereçamento a Byte

- Para usar ***lw*** - o processador precisa calcular o endereço de destino
- O deslocamento que está no campo da instrução diz quantas palavras se deve avançar ou recuar
- Como a memória é endereçada por byte, **precisamos converter palavras em bytes**
- Cada palavra (**WORD**) possui **4 bytes**, por isso multiplicamos o deslocamento por 4 o nosso deslocamento
- Logo, **$A[8]$** corresponde a **$A[4 * 8 + \text{endereço-base}]$** ou **$A[32 + \text{endereço-base}]$**
- Ou seja, se tivermos **$A[i]$** teremos **$A[4 * i + \text{endereço-base}]$**

Endereçamento a Byte (cont.)

Deslocamento (offset) no MIPS

Memória endereçada por byte; palavra = 4 bytes



lw / sw — deslocamento já em bytes

endereço = base + deslocamento

lw \$t0, 8(\$s1) → endereço_base + 8 × 4

COMPILAÇÃO USANDO UMA VARIÁVEL PARA INDEXAR O ARRAY

- Com base no exemplo de um array indexado por uma variável:

```
g = h + A[i];
```

- Suponha que *A* é um array de 100 elementos cujo endereço-base está no registrador **\$s3** e que o compilador associa as variáveis *g*, *h* e *i* aos registradores **\$s1**, **\$s2** e **\$s4**. Qual o código gerado para o MIPS, correspondente ao comando C acima?

COMPILAÇÃO USANDO UMA VARIÁVEL PARA INDEXAR O ARRAY - Solução

- Antes de carregar o valor de **$A[i]$** é necessário achar o endereço
- Antes de somar o valor i ao endereço-base do Array A é necessário multiplicar o valor do índice i por 4
 - $i + i = 2i$ ∴ $2i + 2i = 4i$ -> temos uma multiplicação 4x

```
add $t1, $s4, $s4 # $t1 recebe 2*i  
add $t1, $t1, $t1 # $t1 recebe 4*i
```

- Agora é necessário somar **\$t1** ao endereço-base (armazenado em **\$s3**)

```
add $t1, $t1, $s3 # $t1 recebe (4*i + $s3)
```

COMPILAÇÃO USANDO UMA VARIÁVEL PARA INDEXAR O ARRAY - Solução (cont.)

- Agora é possível usar esse endereço para carregar o valor contido em $A[i]$ em um registrador temporário:

```
lw $t0, 0($t1) # $t0 recebe o conteúdo de A[i]
```

- Por fim, a instrução final do código MIPS para o comando C soma o valor de $A[i]$ com h , e coloca o resultado em g :

```
add $s1, $s2, $t0 # g recebe h + A[i]
```

COMPILAÇÃO USANDO UMA VARIÁVEL PARA INDEXAR O ARRAY - Solução (cont.)

- Imagine então o código acima dentro de um laço ***for (){}***

```
vamos supor $s4 = i
for (i=0;i<3; i++){ // ilustração

    add $t1, $s4, $s4 # $t1 recebe 2*i
    add $t1, $t1, $t1 # $t1 recebe 4*i
    add $t1, $t1, $s3 # $t1 recebe (4*i + $s3)

    lw $t0, 0($t1) # $t0 recebe o conteúdo de A[i]
    ...
}
```

Aqui
conseguimos
“navegar” na
memória

Resumo - Operandos do MIPS

NOME	EXEMPLO	COMENTÁRIOS
32 Registradores	\$s0, \$s1, \$s2... \$t0, \$t1, \$t2... \$v0, \$zero, \$at, \$sp, pc...	Posições de acesso rápido para armazenamento de dados. No MIPS, os dados devem estar em registradores para que as operações aritméticas possam ser realizadas.
2³⁰ palavras de memória	Mem[0], Mem[4],... Mem[4294967292]	No MIPS, essas posições só são acessadas por instruções de transferência de dados. O MIPS endereça byte, de modo que endereços de palavras consecutivas diferem de 4 unidades.

Resumo - Linguagem de Montagem do MIPS

Categoria	Instrução	Exemplo	Significado	Comentário
ARITMÉTICA	add	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Três operandos; dados em registradores
	subtract	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Três operandos; dados em registradores
TRANS. DADOS	load word	lw \$t1, 100 (\$s2)	$\$t1 = \text{Mem}[\$s2+100]$	Dados -> memória para registradores
	store word	sw \$t1, 100 (\$s2)	$\text{Mem}[\$s2+100] = \$t1$	Dados -> registradores para memória

Resumo - Linguagem de Máquina do MIPS

MIPS machine language

Nome	Formato	Example						Comentários
add	R	0	18	19	17	0	32	add \$s1, \$s2, \$s3
sub	R	0	18	19	17	0	34	sub \$s1, \$s2, \$s3
lw	I	35	18	17	100			lw \$s1, 100(\$s2)
sw	I	43	18	17	100			sw \$s1, 100(\$s2)
Tamanho do campo		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Todas as instruções MIPS têm 32 bits
Formato R	R	op	rs	rt	rd	shamt	funct	Formato de instrução aritmética
Formato I	I	op	rs	rt	address			Formato de transferência de dados

EXERCÍCIOS

Exercício 1

1. Converta a seguinte instrução MIPS para linguagem de máquina - formato R
($\$s0$ = registrador 16, $\$s1$ = registrador 17, $\$t0$ = registrador 8, opcode R = 0, funct add = 32):
add \$t0, \$s0, \$s1
2. Converta para linguagem de máquina ($\$s2$ = 18, $\$s3$ = 19, $\$t1$ = 9, opcode R = 0, funct de sub = 34):
sub \$t1, \$s2, \$s3
3. Converta para linguagem de máquina (opcode lw = 35, $\$s0$ = 16, $\$t0$ = 8, endereço/imediato = 8):
lw \$t0, 8 (\$s0)
4. Converta para linguagem de máquina (opcode sw = 43, $\$s1$ = 17, $\$t1$ = 9, endereço/imediato = 12)
sw \$t1, 12 (\$s1)

Exercício 2

1. Preencha os campos de cada instrução:

```
lw $t0, 0($s0)    # carrega A
lw $t1, 4($s0)    # carrega B
add $t2, $t0, $t1 # t2 = A + B
sw $t2, 8($s0)    # armazena em C
```

INSTRUÇÃO	OP	RS	RT	RD/ADDRESS	SHAMT	FUNCT
lw \$t0, 0(\$s0)						
lw \$t1, 4(\$s0)						
add \$t2, \$t0, \$t1						
sw \$t2, 8(\$s0)						

Exercício 3

1. Preencha os campos de cada instrução:

```
lw $t3, 4($s7)    # carrega A
lw $t4, 8($s7)    # carrega B
sub $t5, $t3, $t4 # t5 = A - B
sw $t5, 12($s7)   # armazena em C
```

INSTRUÇÃO	OP	RS	RT	RD/ADDRESS	SHAMT	FUNCT
lw \$t3, 4(\$s7)						
lw \$t4, 8(\$s7)						
sub \$t5, \$t3, \$t4						
sw \$t5, 12(\$s7)						

Exercício 4

1. Decifre o que faz o seguinte código em binário e escreva em assembly:

```
000000 10000 10001 01000 00000 100000  
100011 10010 01001 00000000000000100
```

INSTRUÇÃO 1:

INSTRUÇÃO 2:

Exercício 5

1. A memória contém os valores: A = 5 (em 0), B = 3 (em 4), C = 10 (em 8). Execute o código abaixo passo a passo e diga o valor final armazenado em cada registrador e na memória após a execução:

```
lw    $t0, 0($s0)
lw    $t1, 4($s0)
sub   $t2, $t0, $t1
sw    $t2, 12($s0)
lw    $t3, 8($s0)
sub   $t4, $t3, $t2
sw    $t4, 16($s0)
```

REGISTRADORES: \$t0= ; \$t1= ; \$t2= ; \$t3= ; \$t4=

MEMÓRIA: mem[0]= ; mem[4]= ; mem[8]= ; mem[12]= ; mem[16]=
;

Exercício 6

1. O seguinte valor binário de 32 bits representa uma instrução MIPS:

101011 10001 01001 00000000000001000

Qual é o formato (R ou I)?

Qual é a instrução em assembly?

O que ela faz?

Síntese dos Principais Pontos

- **Instruções Aritméticas**
 - **add** - instrução de soma
 - **sub** - instrução de subtração
 - **Load word** - busca um dado em uma posição da memória
 - **Store Word** - armazena um dado em uma posição da memória